



復旦大學
FUDAN UNIVERSITY

算法设计与分析（第三讲）

黄 橙

计算与智能创新学院

chuang@fudan.edu.cn

课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

求解复杂算法问题

线性规划，在线算法，AI算法

理解高效计算的边界

P vs NP, NP 完全性，近似算法

随机算法

- 随机算法是在逻辑或执行过程中引入随机性（例如随机数）来帮助求解问题的一类计算机算法
- 为什么需要在算法中引入随机？



资源竞争场景

- 两个人同时抢占一个资源，会发生什么：假设两个进程 P_1, P_2 同时想发送数据

确定性策略

两者同时发送



发生冲突



都等待 1 秒



再次同时发送



再次冲突



问题：

如果两个进程执行完全相同的确定性规则，

相同状态 + 相同规则 $\Rightarrow$ 相同动作

它们可能不断重复同样的冲突

网络随机退避

如果加入一点随机性呢？

发生冲突后：

- P_1 ：随机等待一段时间
- P_2 ：也随机等待一段时间

例如：

$$P_1 \rightarrow 0.3s, P_2 \rightarrow 0.8s$$

于是 P_1 先发送成功，冲突被解除。

核心思想

随机性可以打破对称性

当多个参与者处于相同状态、执行相同规则时，
确定性算法可能让它们一直同步失败；
而随机化可以让它们走向不同的选择，从而摆脱僵局。

为什么需要在算法中引入随机

- 避免固定策略被最坏输入针对
例：随机快速排序。
 - 避免为了找到最优选择付出太大代价
例：随机选 pivot，不必每次精确寻找中位数。
 - 允许极小错误概率，大幅减少计算
例：Freivalds 矩阵乘法验证。
 - 打破对称性，避免多个参与者陷入相同决策
例：网络随机退避、分布式协议中的随机选择。
 - 让算法设计更简单
例：Miller-Rabin 素数测试。
- 随机性让单次行为变得不可预测，但让整体性能可以用概率严格控制。

随机算法 vs. 确定性算法

- 对于相同的输入，在不同的执行过程中
 - 随机算法可能会运行不同数量的步骤
 - 随机算法产生不同的输出
- 算法在不同的执行过程中，会使用不同的随机数，而这些随机数可能会改变执行的过程

随机算法的两种类型

- 蒙特卡罗算法（Monte Carlo Algorithm）：对于每一个输入，
 - 无论抛掷的硬币如何，算法**总是以多项式时间运行**。
 - 输出正确的概率大于某个较高的值（**不是100%**）。
 - 案例：矩阵乘法验证，Miller-Rabin 素数测试
- 拉斯维加斯算法（Las Vegas Algorithm）：对于每一个输入，
 - 无论抛掷的硬币如何，算法**都是正确的**。
 - 算法在**期望的多项式时间**内运行。
 - 案例：随机快速排序

随机的来源

- 伪随机数生成器
 - 线性同余生成器: $X_{n+1} = (aX_n + c) \bmod m$
 - 密码算法: Hash算法, 对称加密算法
- 现实随机来源
 - 放射性衰变
 - 电磁噪声, 大气噪声

蒙特卡罗算法的主要特性

- 运行时间：多项式时间内运行(最坏情况下)
 - 也就是说，对于长度为 n 的输入，算法的运行时间是 $O(n^k)$ 的形式，其中 k 是个常数
- 正确性：在**高概率下**是正确的
 - 常数概率 $\geq \frac{1}{2}$
 - 多项式高概率 $\geq 1 - \frac{1}{n^c}, \exists c, n_0 > 0, \forall n > n_0$
 - 指数高概率 $\geq 1 - \frac{1}{2^{nc}}, \exists c, n_0 > 0, \forall n > n_0$
- 概率提升：
 - 通过重复算法多次，可以将常数概率提升到指数高概率

矩阵乘法验证

- 计算矩阵乘法
 - 时间复杂度为 $O(n^3)$
 - Strassen算法：使用 7 次乘法来实现两个的矩阵相乘，时间复杂度为 $O(n^{2.81})$
- 验证矩阵乘法
 - 输入： $n * n$ 的矩阵， A, B, C
 - 目标：验证 $C = A \times B$?

Freivalds算法

1. 选择一个随机的二进制 n -位向量 $r = r_1 \dots r_n$, 每个 r_i 是独立的, $i = 1 \dots n$ ($\Pr[r_i = 1] = \frac{1}{2}$)
 2. 如果 $A(Br) = Cr$, 则输出 'Yes', 否则输出 'No'
- **分析: 时间复杂度** $O(n^2)$
 - 需要进行3次矩阵-向量乘法: Br 、 $A(Br)$ 、 Cr , 每次乘法的复杂度是 $O(n^2)$
 - **正确性:** 当 $AB = C$ 时, 对于所有的 r , 有 $A(Br) = (AB)r = Cr$
 - 当 $AB = C$ 时, 算法总是输出YES, 即 $\text{Prob}[\text{错误}] = 0$
 - 如果 $AB \neq C$, $\Pr[ABr = Cr]$ 是多少?

错误率分析

定理 设 A, B, C 为 $n \times n$ 矩阵, 随机向量 $r = (r_1, \dots, r_n)^T \in \{0, 1\}^n$ 的每个分量独立且等概率取 0 或 1。如果 $AB \neq C$, 则 $\Pr[A(Br) = Cr] \leq \frac{1}{2}$ 。

- 令 $D = AB - C$, 由于 $AB \neq C$, 因此 $D \neq 0$ 。所以 D 中至少存在一个非零元素。取某个位置 (i, j) , 使得 $D_{ij} \neq 0$ 。注意到 $A(Br) = Cr \Leftrightarrow (AB - C)r = 0 \Leftrightarrow Dr = 0$, 因此我们只需要证明 $\Pr[Dr = 0] \leq \frac{1}{2}$ 。
- **证明:** 考虑向量 Dr 的第 i 个分量: $(Dr)_i = \sum_{k=1}^n D_{ik} r_k = D_{ij} r_j + \sum_{k \neq j} D_{ik} r_k$ 。现在固定除 r_j 以外的所有随机变量 $r_k, k \neq j$ 。在这些变量固定以后, 令 $S = \sum_{k \neq j} D_{ik} r_k$, 则 S 是一个固定值, 于是 $(Dr)_i = D_{ij} r_j + S$ 。由于 $r_j \in \{0, 1\}$, 只有两种可能:
 - 当 $r_j = 0$ 时, $(Dr)_i = S$; 当 $r_j = 1$ 时, $(Dr)_i = S + D_{ij}$ 。
 - 因为 $D_{ij} \neq 0$, 所以 S 与 $S + D_{ij}$ 不可能同时等于 0。因此, 在 $r_j = 0$ 和 $r_j = 1$ 两种取值中, 至多有一种能够使 $(Dr)_i = 0$ 。又因为 r_j 独立且均匀地取 0 或 1, 所以对于任意固定的其他 $n - 1$ 个分量, 都有 $\Pr[(Dr)_i = 0 \mid \{r_k: k \neq j\}] \leq \frac{1}{2}$ 。对其他 $n - 1$ 个随机变量取平均, 得到 $\Pr[(Dr)_i = 0] \leq \frac{1}{2}$ 。
- 最后, 由于 $Dr = 0 \Rightarrow (Dr)_i = 0$, 所以事件 $\{Dr = 0\} \subseteq \{(Dr)_i = 0\}$ 。因此 $\Pr[Dr = 0] \leq \Pr[(Dr)_i = 0] \leq \frac{1}{2}$ 。即

$$AB \neq C \Rightarrow \Pr[A(Br) = Cr] \leq \frac{1}{2}$$

素数判定问题

- 判断方法（试除法）

- 对于大于1的整数 n ：求平方根：计算 $\sqrt{n}$ 依次试除：用从2到 $\sqrt{n}$ 之间的所有整数去除 n
- 得出结论：如果能被其中任何一个整数整除，它就不是质数（是合数）；如果都不能整除，它就是质数。

- 复杂度分析

- 时间复杂度： $O(\sqrt{n})$

- 最坏情况：当数字 n 是一个质数时，程序必须从2一直除到 $\lfloor \sqrt{n} \rfloor$ （向下取整）。总共需要尝试的除法次数大约为 $\sqrt{n} - 1$ 次。

模运算：只保留余数

符号或规则	例子
$a \equiv b \pmod{n}$	$100 \equiv 1 \pmod{11}$
$-1 \equiv n - 1 \pmod{n}$	$-1 \equiv 16 \pmod{17}$
先取模，再乘法取模	$(20 \times 21) \pmod{17} = (3 \times 4) \pmod{17} = 12$

如果 n 是素数，有没有一个很容易验证的必要条件？

- Fermat 检验： $p \text{ prime} \Rightarrow a^{p-1} \equiv 1 \pmod{p}$ ($\gcd(a, p) = 1$)
- 随机化算法（判断 n 是否为素数）：
 - 输入 n
 - 随机选择 $a \in \{2, 3, \dots, n-1\}$
 - 计算 $a^{n-1} \pmod{n}$
 - 如果结果为1，**可能是素数？**
 - $561=3 \times 11 \times 17$ 是合数。
 - 但： $a^{560} \equiv 1 \pmod{561}$
 - 对于所有 $\gcd(a, 561) = 1$ 都成立
 - 如果结果不为1，一定是合数

$$n \text{ 是素数} \Rightarrow a^{n-1} \equiv 1 \pmod{n}$$

取逆否命题：

$$a^{n-1} \not\equiv 1 \pmod{n} \Rightarrow n \text{ 不是素数}$$

而 $n > 1$ ，不是素数就意味着它是合数。

素数还有一个更强的结构性质

$$x^2 \equiv 1 \pmod{p}$$

推出：

$$(x-1)(x+1) \equiv 0 \pmod{p}$$

由于 p 是素数：

$$x \equiv 1 \text{ or } x \equiv -1$$

得到：

模素数 p ，平方根只有 ± 1



如果 n 是素数，那么

$$a^{n-1} \equiv 1 \pmod{n}.$$

如果 n 是奇数，那么 $n-1$ 一定是偶数。

比如 $n = 41$ ：

$$a^{40} = (a^{20})^2.$$

所以 Fermat 告诉我们：

$$(a^{20})^2 \equiv 1 \pmod{n}.$$

如果 n 真的是素数，那么“平方以后等于 1”的数只能是：

$$a^{20} \equiv 1$$

或者

$$a^{20} \equiv -1.$$

这就多获得了额外信息

反例：取 $n = 15, x = 4$ ，那么： $4^2 = 16 \equiv 1 \pmod{15}$

所以 4 是模 15 下 1 的平方根。

可是： $4 \neq 1, 4 \neq -1 \equiv 14 \pmod{15}$

于是找到了： $4^2 \equiv 1 \pmod{15}, 4 \neq \pm 1$

这叫做 1 的非平凡平方根

进一步分析这个性质

- 因为 $a^{20} = (a^{10})^2$, 所以如果 $a^{20} \equiv 1$, 那么对于素数 n , 又必须有: $a^{10} \equiv \pm 1$.
- 如果它还是 1, 继续: $a^{10} = (a^5)^2$ 。因此继续问: $a^5 \equiv \pm 1$?
- 一般化:

- 把 $n - 1$ 里面所有的 2 都提出来。例如: $40 = 2^3 \times 5$, 于是: $a^{40} = \left(\left((a^5)^2 \right)^2 \right)^2$.

- 也就是: $a^5 \rightarrow a^{10} \rightarrow a^{20} \rightarrow a^{40}$.

- 每一步都是: 前一个数平方得到后一个数

- 得到:

$$n - 1 = 2^s d, d \text{ 为奇数}$$

- 能否基于这个性质找到素数?

判断素数

$$a^d \xrightarrow{\text{平方}} a^{2d} \xrightarrow{\text{平方}} a^{4d} \rightarrow \dots \xrightarrow{\text{平方}} a^{2^s d} = a^{n-1}$$

假如发现：

$$x^2 \equiv 1 \pmod{n}$$

但同时：

$$x \not\equiv \pm 1 \pmod{n}.$$

如果 n 是素数，这是不可能的。

由逆否命题得到：

$$\boxed{\exists x: x^2 \equiv 1 \pmod{n}, x \not\equiv \pm 1 \Rightarrow n \text{ 一定是合数}}$$

因为： $561 - 1 = 560 = 2^4 \times 35$.

所以先算：

$$2^{35}$$

然后不断平方：

$$2^{35} \rightarrow 2^{70} \rightarrow 2^{140} \rightarrow 2^{280} \rightarrow 2^{560}$$

模 561 后： $263 \rightarrow 166 \rightarrow 67 \rightarrow 1 \rightarrow 1$

所以不是素数

快速模指数算法

- 核心思想很简单：

$$a^{2k} = (a^k)^2 \bmod n$$

$$a^{2k+1} = a \cdot (a^k)^2 \bmod n$$

- 所以每次把指数 d 除以 2，就能把问题规模减半。
- 例如计算： $3^{13} \bmod 17$
 - 通过不断平方：
 - $3^1 \equiv 3, 3^2 \equiv 9, 3^4 \equiv 9^2 = 81 \equiv 13 \pmod{17}, 3^8 \equiv 13^2 = 169 \equiv 16 \pmod{17}$
- 于是： $3^{13} \equiv 3^8 \cdot 3^4 \cdot 3 \equiv 16 \cdot 13 \cdot 3 \equiv 12 \pmod{17}$.

Miller-Rabin素数判定算法

MillerRabin(n, k):

处理简单情况

if $n == 2$ or $n == 3$:

 return PRIME

if $n < 2$ or n is even:

 return COMPOSITE

将 $n - 1$ 分解成 $2^s * d$, 其中 d 为奇数

write $n - 1 = 2^s * d$

repeat k times:

 # 随机选择一个底数

 choose random a from $\{2, 3, \dots, n - 2\}$

 # 计算平方链的起点

$x = a^d \bmod n$

 # 情况 1: 起点就是 1 或 -1

 if $x == 1$ or $x == n - 1$:

 continue # 本轮通过

 # 不断平方

 for $r = 1$ to $s - 1$:

$x = x^2 \bmod n$

 # 出现 -1: 本轮通过

 if $x == n - 1$:

 break

 # 可选: 出现非平凡平方根

 if $x == 1$:

 return COMPOSITE

 # 如果整个平方过程中都没有出现 -1

 if $x != n - 1$:

 return COMPOSITE

return PROBABLY_PRIME

Miller-Rabin 的时间复杂度

$$n - 1 = 2^s d, d \text{ 为奇数}$$

- 一次测试主要包含： $x = a^d \bmod n$ 运算，以及最多 $s - 1$ 次 $x \leftarrow x^2 \bmod n$ 运算
- 每轮需要 $O(\log n)$ 次模乘
- 如果采用普通的大整数乘法： $O((\log n)^2)$
- 因此：每轮时间复杂度 $O((\log n)^3)$
- 重复 k 轮： $O(k(\log n)^3)$

因为：

$$n - 1 = 2^s d.$$

取以 2 为底的对数：

$$\log_2(n - 1) = s + \log_2 d.$$

因此：

$$O(\log d) + O(s) = O(\log d + s) = O(\log n)$$

大整数乘法的计算开销

设两个整数 x, y 都有 L bit:

$$x = (x_{L-1} \cdots x_1 x_0)_2$$

$$y = (y_{L-1} \cdots y_1 y_0)_2$$

采用普通“竖式乘法”，每一位都要和另一数的 L 位相乘：

$$\underbrace{L}_{x \text{ 的位数}} \times \underbrace{L}_{y \text{ 的位数}}$$

因此：

$$T_{\text{mul}}(L) = O(L^2)$$

例如：

$$x = 1101_2 = 13, y = 1011_2 = 11$$

做普通二进制竖式乘法： $13 \times 11 = 143$

1101	
$\times 1011$	

1101	$\leftarrow 1 \times 1101$
1101	$\leftarrow 1 \times 1101$, 左移1位
0000	$\leftarrow 0 \times 1101$, 左移2位
+ 1101	$\leftarrow 1 \times 1101$, 左移3位

10001111	

正确性分析

- 对任意奇合数 n , Rabin 证明:

$$\text{Strong Liars的数量} \leq \frac{1}{4} \times \text{所有候选底数}$$

- 因此随机选择 a : $P(\text{误判}) \leq \frac{1}{4}$, $P(\text{发现合数}) \geq \frac{3}{4}$

- 独立重复 k 轮: $P(\text{连续被骗}) \leq \left(\frac{1}{4}\right)^k$

拉斯维加斯算法的主要特性

- 可以在随机的情况下生成一个 $r \in \{0, \dots, R\}$
- 对于输入 x ，运行时间变成一个随机变量 $Time(x, r)$ ，它依赖于 r
- **算法运行是多项式时间的期望：** 对任意长度为 n 的输入 x ，期望时间 $T(n) = E_r[Time(x, r)]$ ，被一个关于 n 的多项式函数所界定
- **算法结果总是正确：** 对于所有输入， $Prob[\text{错误}] = 0$

如何选择快速排序算法的区分点？

- 基础快速排序: $x = A[l]$
 - 最坏时间复杂度: $O(n^2)$
- 引入随机化: 随机快速排序
 - 从数组 A 中随机选择区分点 x (递归过程中)

随机快速排序算法

```
RAND-PARTITION( $A, l, r$ ) //数组 $A[l, \dots, r]$   
   $i \leftarrow \text{RANDOM}(l, r)$   
  exchange  $A[l]$  with  $A[i]$   
  return PARTITION( $A, l, r$ )
```

```
RAND-QUICKSORT ( $A, l, r$ ) //数组 $A[l, \dots, r]$   
  if  $l < r$  then  
     $x \leftarrow \text{RAND-PARTITION}(A, l, r)$   
    QUICK-SORT( $A, l, x-1$ )  
    QUICK-SORT( $A, x+1, r$ )
```

因为随机性：没有特定的输入会导致最坏情况的行为

概率论基础回顾

- **样本空间：** 是一个包含所有可能结果的集合 S
 - 例如：所有可能的区分点序列
- **概率事件：** 是从样本空间中选出的一个或多个可能结果的集合，是样本空间的一个子集
 - 例如：一个集合包括 区分点序列 实现 1/4:3/4 的区分
- **随机变量：** 是一个函数 X ，将样本空间 S 中的每一个可能结果与一个实数值 $\mathbb{R}$ 关联, $X: S \rightarrow \mathbb{R}$ 。
 - 例如：随机快速排序 中递归过程 子数组 的大小
- **随机变量的期望：** X 的平均值，以概率为权重 $E(X) = \sum_{i \in S} X_i \cdot \text{Pr}(i)$
 - 例如：随机输入情况下，随机快速排序中第一个递归子数组的平均大小

概率论基础回顾

- **期望的线性性质：** 设是 $X_1, \dots, X_n$ 是定义在 S 上的随机变量。那么 $E[\sum X_i] = \sum E[X_i]$ ，对于所有随机变量，这个性质都成立。
- **事件独立：** 如果对于事件 A 和 B ($A, B \subseteq S$)，有 $\Pr(AB) = \Pr(A) * \Pr(B)$ ，则 A 和 B 是独立事件。
- **随机变量独立：** X 和 Y 独立，则
 - $\Pr(X = a, Y = b) = \Pr(X = a) * \Pr(Y = b)$
 - $E(X * Y) = E(X) * E(Y)$

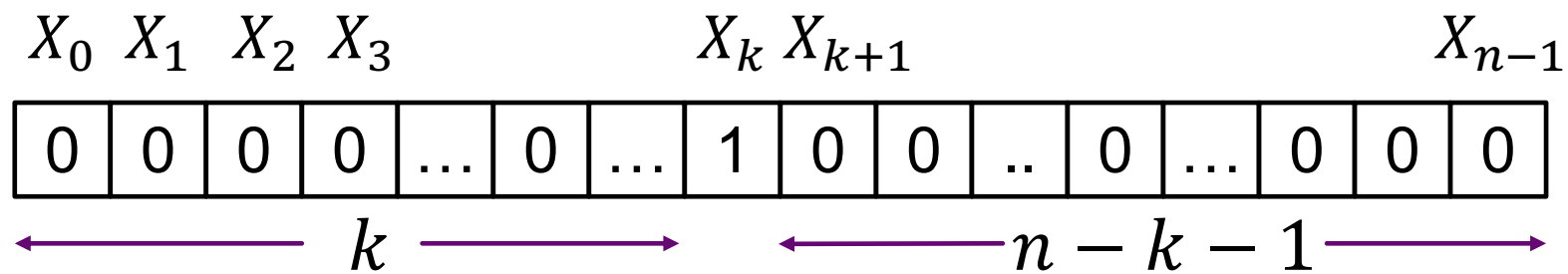
随机快速排序算法分析

- **样本空间：**在随机快速排序算法执行中，所有可能的序列 S （在有区分点的情况下）
- 设 $T(n)$ 是一个随机变量，表示随机快速排序在大小为 n 的输入上的运行时间
 - $T(n): S \rightarrow \mathbb{R}$: （任意）选择一组区分点会产生一个运行时间。
- 目标：计算 $E[T(n)]$

指示器随机变量

- 定义一个指示器随机变量, $k = 0, 1, 2, \dots, n - 1$

$$X_k = \begin{cases} 1 & \text{选择的区分点得到一个 } k:n-k-1 \text{ 划分} \\ 0 & \text{其他} \end{cases}$$



指示器随机变量的期望

- 随机变量 X_k 的期望

$$E[X_k] = 1 * \Pr\{X_k = 1\} + 0 * \Pr\{X_k = 0\} = 1 * \Pr\{X_k = 1\} = 1/n$$

- 意味着：所有的划分都是等可能的
- 用 $E[X_k]$ 来帮助计算 $E[T(n)]$
- 用 X_k 将所有 n 种情况总结在一个表达式中

$$T(n) = \begin{cases} T(0) + T(n-1) + \Theta(n) & 0:n-1 \text{ 划分} \\ T(1) + T(n-2) + \Theta(n) & 1:n-2 \text{ 划分} \\ T(3) + T(n-3) + \Theta(n) & 2:n-3 \text{ 划分} \\ \dots & \dots \\ T(n-2) + T(1) + \Theta(n) & n-2:1 \text{ 划分} \\ T(n-1) + T(0) + \Theta(n) & n-1:0 \text{ 划分} \end{cases}$$



$$T(n) = \sum_{k=0}^{n-1} X_k (T(k) + T(n-k-1) + \Theta(n))$$

时间复杂度的期望

- $$E[T(n)] = E\left[\sum_{k=0}^{n-1} X_k (T(k) + T(n-k-1) + \Theta(n))\right] = \sum_{k=0}^{n-1} E[X_k (T(k) + T(n-k-1) + \Theta(n))] =$$
$$\sum_{k=0}^{n-1} E[X_k] * E[T(k) + T(n-k-1) + \Theta(n)] = \frac{1}{n} \sum_{k=0}^{n-1} E[T(k)] + \frac{1}{n} \sum_{k=0}^{n-1} E[T(n-k-1)] +$$
$$\frac{1}{n} \sum_{k=0}^{n-1} \Theta(n) = \frac{2}{n} \sum_{k=0}^{n-1} E[T(k)] + \Theta(n)$$
- 性质：
 - 期望的线性性质
 - 在递归调用中，当前层的 X_k 与未来递归调用使用的随机性是独立的。
 - $E(X_k) = 1/n$

时间复杂度分析

- $E[T(n)] = \frac{2}{n} \sum_{k=2}^{n-1} E[T(k)] + \Theta(n)$
- 化简: $k = 0, 1$ 两种情况, 可以包含在 $\Theta(n)$
- **$E[T(n)] \leq cn \log n$, c 是常数**

递推:

$$E[T(n)] \leq \frac{2}{n} \sum_{k=2}^{n-1} E[T(k)] + an.$$

归纳假设

$$E[T(k)] \leq ck \log k, k < n.$$

因此:

$$E[T(n)] \leq \frac{2c}{n} \sum_{k=2}^{n-1} k \log k + an.$$

利用:

$$\sum_{k=2}^{n-1} k \log k \leq \int_1^n x \log x \, dx = \frac{n^2}{2} \log n - \Theta(n^2),$$

得到:

$$E[T(n)] \leq cn \log n - \Theta(cn) + an.$$

取足够大的 c , 使

$$\Theta(cn) \geq an,$$

于是:

$$\boxed{E[T(n)] \leq cn \log n}.$$

选择问题

- 从 n 个元素的集合中选择第 i 小的元素
 - 先排序，再取出第 i 位的元素，最坏时间复杂度： $\Theta(n \lg n)$
- 随机选择算法
 - 运行时间的期望为线性
- 线性时间的选择算法
 - 最坏情况下的时间复杂度为线性

随机选择算法

RAND-SELECT(A, p, q, i) $\triangleright$ $A[p..q]$ 的第 i 小元素

if $p = q$ then

 return $A[p]$

$r \leftarrow \text{RAND-PARTITION}(A, p, q)$

$k \leftarrow r - p + 1$ $\triangleright k = \text{rank}(A[r])$

if $i = k$ then

 return $A[r]$

if $i < k$ then

 return **RAND-SELECT**($A, p, r - 1, i$)

else

 return **RAND-SELECT**($A, r + 1, q, i - k$)

核心步骤是：

$r \leftarrow \text{RAND-PARTITION}(A, p, q)$

随机选 pivot 并完成 partition。然后

$k = r - p + 1$

表示 pivot $A[r]$ 在当前子数组 $A[p..q]$ 中的排名。

三种情况：

$i = k$

说明 pivot 正好就是第 i 小，返回 $A[r]$ 。

如果

$i < k,$

目标在左半边：

$A[p..r - 1]$

所以递归：

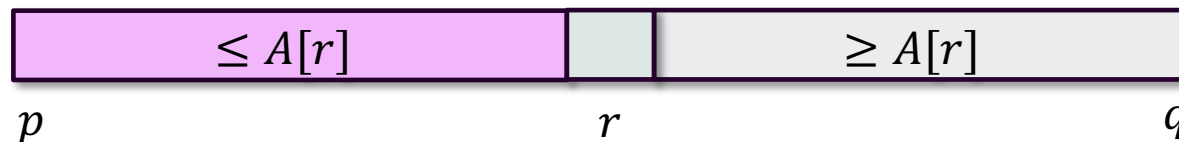
RAND-SELECT($A, p, r - 1, i$).

如果

$i > k,$

目标在右半边：

$A[r + 1..q]$



随机选择算法

目标：在数组 $A = [9, 2, 7, 4, 8, 1, 6, 3, 5]$ 中找到第 5 小元素

1 Step 1: 随机选择 pivot = 7

寻找第 $i = 5$ 小元素



pivot = 7

$A = [9, 2, 7, 4, 8, 1, 6, 3, 5]$

$i = 5$

2 Partition 后, pivot 的排名 $k = 7$



Partition 后得到:

$[2, 4, 1, 6, 3, 5] \mid 7 \mid [9, 8]$

pivot 的排名 $k = 7$ (在整个数组中)

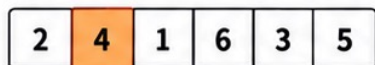
3 因为 $i < k$, 只递归左侧子数组



递归到这个子数组

因为 $i = 5 < k = 7$,
只递归左侧子数组 $[2, 4, 1, 6, 3, 5]$ 。

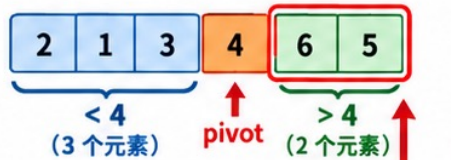
4 在左侧子数组中, 随机选择 pivot = 4 得到 $k = 4$ (相对于当前子数组)



Partition 后得到: $[2, 1, 3] \mid 4 \mid [6, 5]$

pivot 的排名 $k = 4$ (相对于当前子数组)

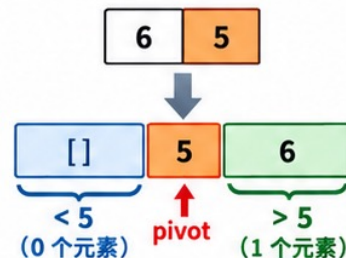
5 因为 $i > k$, 只递归右侧子数组, 并更新 $i \leftarrow i - k = 1$



递归到这个子数组

因为 $i = 5 > k = 4$,
只递归右侧子数组 $[6, 5]$,
并更新 $i \leftarrow i - k = 5 - 4 = 1$
(即在该子数组中寻找第 1 小元素)。

6 在子数组 $[6, 5]$ 中, 选择 pivot = 5



此时 $k = 1$, 且 $i = 1$,
因此返回 pivot = 5。

找到答案: 第 5 小元素 = 5

最好情况和最坏情况的时间复杂度

- 最好情况:

- $T(n) = T(n/2) + \Theta(n)$

- $\Rightarrow T(n) = \Theta(n)$

- 最坏情况:

- $T(n) = T(n - 1) + \Theta(n)$

- $\Rightarrow T(n) = \Theta(n^2)$

时间复杂度的期望

- 设集合中的元素都是互异的
- 设 $T(n)$ 是一个随机变量，表示随机选择算法在大小为 n 的输入集合上的运行时间
- 定义一个指示器随机变量， $k = 0, 1, 2, \dots, n - 1$

$$X_k = \begin{cases} 1 & \text{选择的区分点得到一个 } k:n-k-1 \text{ 划分} \\ 0 & \text{其他} \end{cases}$$

时间复杂度的期望

- 假设第 i 个元素总是落在分区的较大一侧（上界）

$$T(n) = \begin{cases} T(\max(0, n-1)) + \Theta(n) & 0:n-1 \text{ 划分} \\ T(\max(1, n-2)) + \Theta(n) & 1:n-2 \text{ 划分} \\ T(\max(2, n-3)) + \Theta(n) & 2:n-3 \text{ 划分} \\ \dots & \dots \\ T(\max(n-2, 1)) + \Theta(n) & n-2:1 \text{ 划分} \\ T(\max(n-1, 0)) + \Theta(n) & n-1:0 \text{ 划分} \end{cases}$$

$$T(n) = \sum_{k=0}^{n-1} X_k (T(\max(k, n-k-1)) + \Theta(n)) \Rightarrow E[T(n)] = \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + \Theta(n)$$

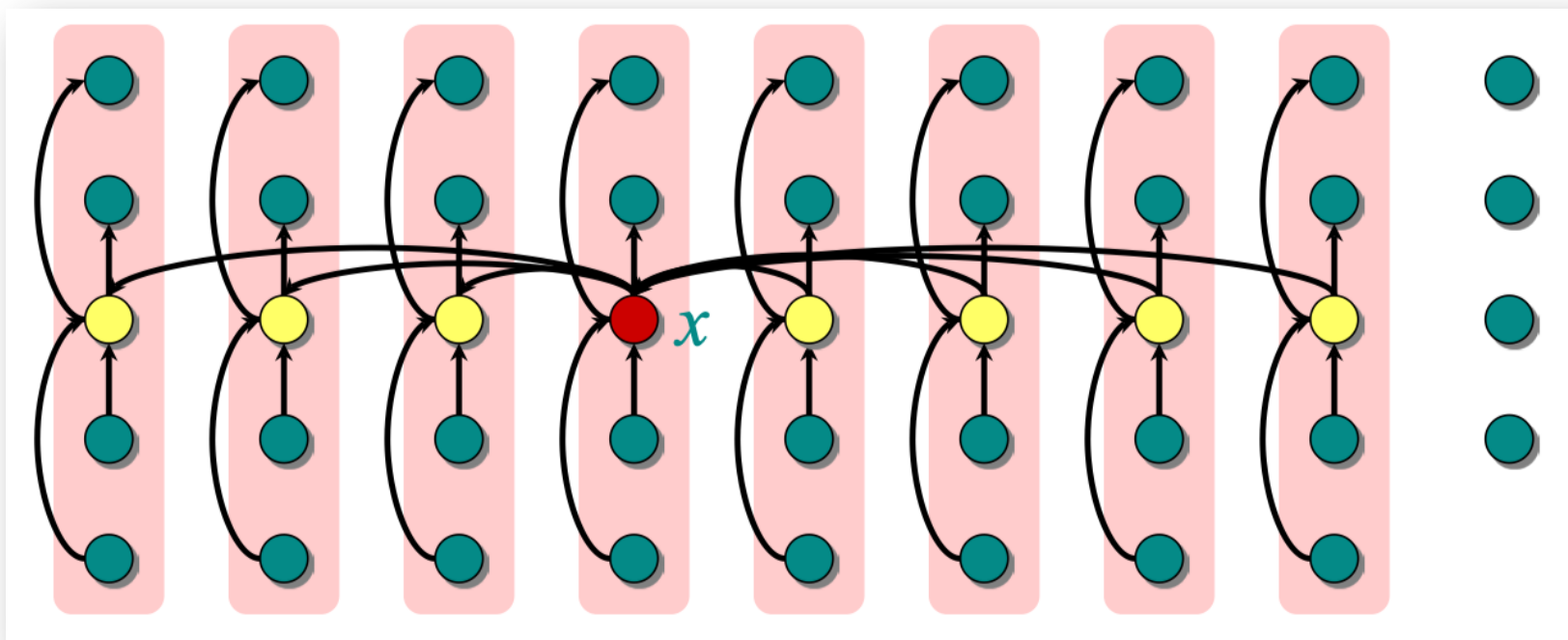
$$E[T(n)] \leq cn, c \text{ 是常数}, E[T(n)] = O(n)$$

实际应用中是非常优秀的算法

线性时间的选择算法

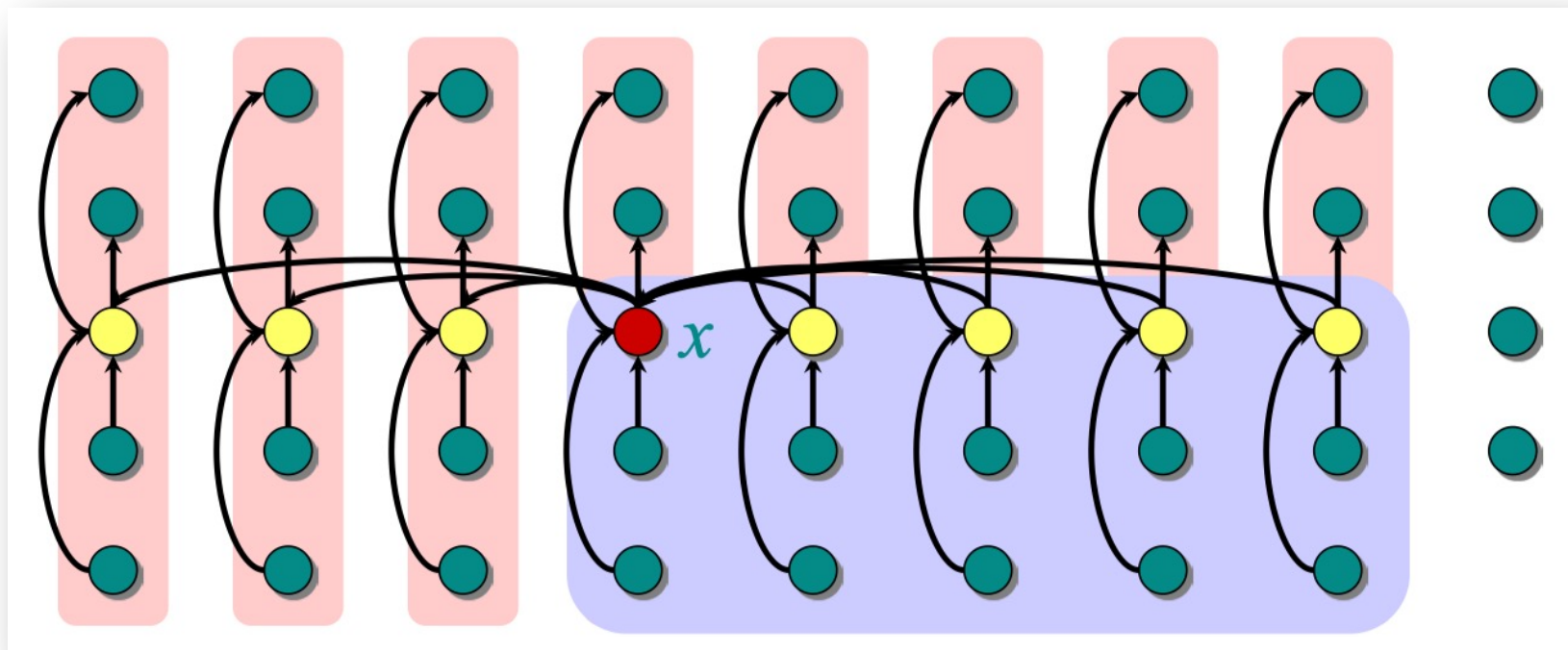
- $SELECT(i, n)$
- 1. 将 n 个元素分成若干个 5 个一组的小组，通过直接排序找到每个小组的中位数。
- 2. 递归地 SELECT 这 $\lceil n/5 \rceil$ 个中位数的中位数 x 作为区分点。
- 3. 以区分点 x 为中心进行划分($k = rank(x)$)
- 4. 如果 $i = k$ ，则返回 x ；
 如果 $i < k$ ，则在下半部分递归地 SELECT 第 i 小的元素；
 否则，在上半部分递归地 SELECT 第 $i - k$ 小的元素。

区分点选择



1. 将 n 个元素分成若干个5个一组的小组，通过直接排序找到每个小组的中位数
2. 选择这 $\lfloor n/5 \rfloor$ 个中位数的中位数 x 作为区分点

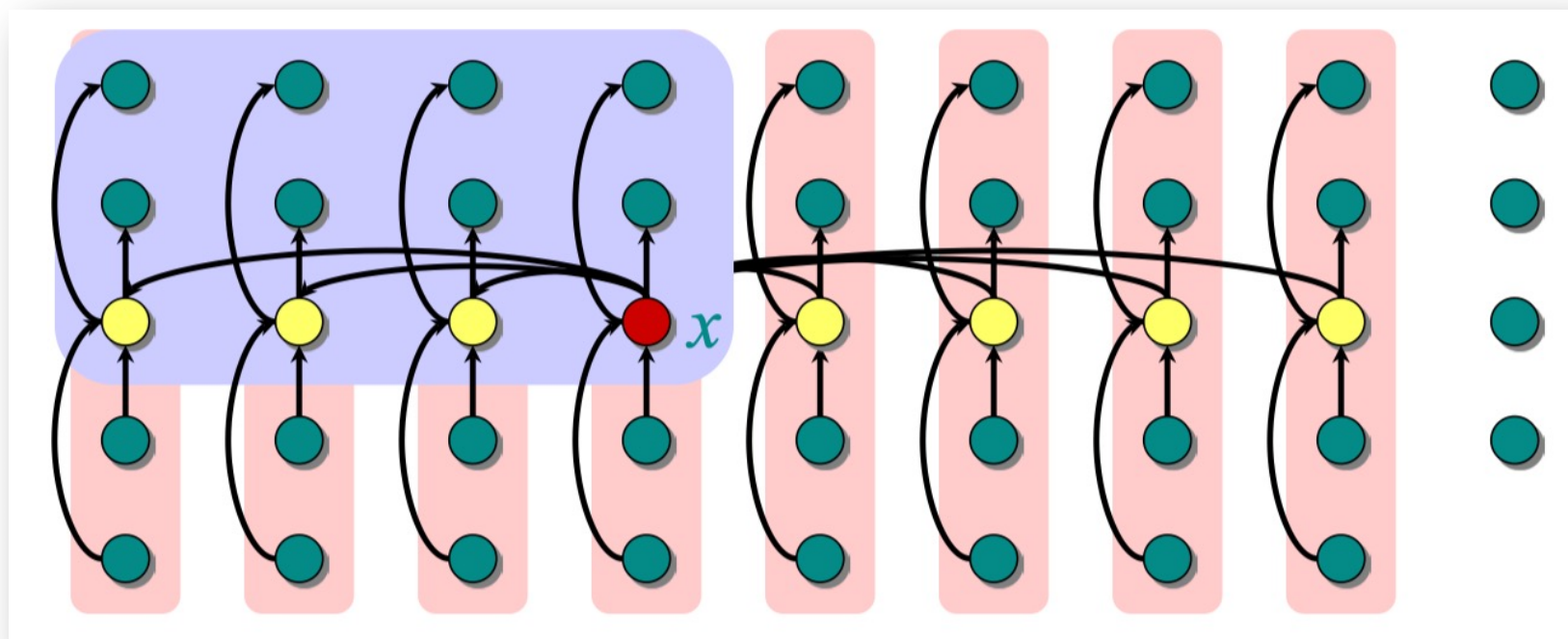
区分点选择



$$3 \left(\left\lfloor \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rfloor \right) \geq \frac{3n}{10}$$

至少有 $\frac{3n}{10}$ 个元素 $\leq x$

区分点选择



$$3 \left(\left\lfloor \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rfloor \right) \geq \frac{3n}{10}$$

至少有 $\frac{3n}{10}$ 个元素 $\leq x$

时间复杂度分析

- SELECT 递归调用在最多 $\leq \frac{7n}{10}$ 个元素上递归执行。
- 运行时间的递归关系可以假设在最坏情况下，步骤 4 的时间为 $T(\frac{7n}{10})$

$O(n)$ 1. 将 n 个元素分成若干个 5 个一组的小组。通过直接排序找到每个小组的中位数。

$T(\frac{n}{5})$ 2. 递归地 SELECT 这 $[n/5]$ 个中位数的中位数 x 作为区分点。

$O(n)$ 3. 以区分点 x 为中心进行划分 ($k = \text{rank}(x)$)

4. 如果 $i = k$ ，则返回 x ；

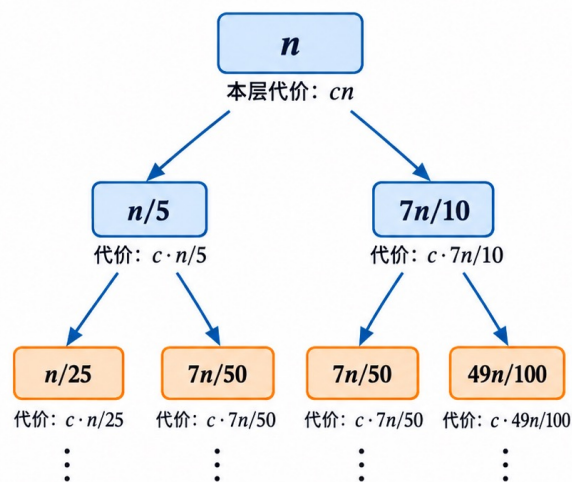
$T(\frac{7n}{10})$ 如果 $i < k$ ，则在下半部分递归地 SELECT 第 i 小的元素；
否则，在上半部分递归地 SELECT 第 $i - k$ 小的元素。

$$T(n) = T(\frac{n}{5}) + T(\frac{7n}{10}) + O(n)$$

时间复杂度分析

- $T(n) \leq cn \left[1 + \frac{9}{10} + \left(\frac{9}{10}\right)^2 + \dots \right]$
- 这是一个等比级数: $\sum_{i=0}^{\infty} \left(\frac{9}{10}\right)^i = \frac{1}{1 - \frac{9}{10}} = 10$
- 所以: $T(n) \leq 10cn$
- 因此: $T(n) = O(n)$

递归树求解: $T(n) = T(n/5) + T(7n/10) + O(n)$



每层总代价	
第0层:	cn
第1层:	$c(n/5 + 7n/10) = c \cdot (9n/10)$
第2层:	$\leq c \cdot (9/10)^2 n$
第 <i>i</i> 层:	$\leq c \cdot (9/10)^i n$

求和得到总复杂度

$$\begin{aligned} T(n) &\leq cn \left[1 + \frac{9}{10} + \left(\frac{9}{10}\right)^2 + \dots \right] \\ &= cn \cdot \frac{1}{1 - 9/10} \\ &= 10cn \\ &= O(n) \end{aligned}$$

结论: $T(n) = \Theta(n)$

因为各层规模和按 9/10 递减, 形成收敛的等比级数。

工程实现上对比

对比维度	随机化 QuickSelect	确定性 BFPRT
期望比较次数	$\sim 3.4n$	$\sim 20-40n$
最坏情况	$O(n^2)$ (概率极低)	$O(n)$ (严格保证)
缓存友好性	好 (顺序访问)	差 (分组+递归开销)
实现复杂度	简单	复杂
实际运行时间	快	慢 2-5 倍
可被对手攻击	不能 (随机化)	不能 (确定性)

分布式系统中的随机化

- 共识算法中的随机化: Ben-Or 协议
- 负载均衡: Power of Two Choices

共识算法中的随机化：Ben-Or 协议

- FLP 不可能性定理：在完全异步、允许至少一个进程崩溃的系统里，任何确定性共识协议，都存在一种合法的消息调度，使系统永远无法决定
- Ben-Or的随机化共识协议绕过了这个不可能性

三个将军：A, B, C，要决定：

0 = 撤退, 1 = 进攻

- 但信使送消息的时间没有上限，而且允许一个将军突然失联。
- A 收不到 C 的消息：C 是死了，还是信使只是堵在路上？A 无法知道。
- 如果 A 必须等 C：C 真死了 → 永远等下去 → 无法继续协议
- 如果 A 不等 C：C 的消息下一秒突然到达，而且里面的信息可能改变整个系统应当做出的决定。

Ben-Or 协议可以很简洁地表达：

先看能不能靠多数达成一致；不行就随机选，再来一轮。

每一轮做三件事：

- **收集大家当前的值**：如果看到某个值 v 有明显多数，就倾向 v 。
- **尝试决定**：如果对 v 的支持已经足够强，就直接决定 v 。
- **还不能决定，就随机化**：如果大家还是分散，就随机选 0 或 1，进入下一轮。

多数能解决就解决；解决不了就掷硬币打破僵局。

每一轮都有非零概率把系统推入可收敛状态

只要每轮成功概率 $p > 0$ ，连续 k 轮都失败的概率大致是： $(1 - p)^k$ ，而： $\lim_{k \rightarrow \infty} (1 - p)^k = 0$ ，因此最终会达成共识的概率是：1

负载均衡

- 问题：n 个球（任务）投入 n 个桶（服务器），如何最小化 最大负载？
- 完全随机：每个球随机选一个桶。最大负载为 $\Theta(\log n / \log \log n)$ 。
- 单个桶的平均负载是 1，因为有 n 个球，每个球以 $1/n$ 的概率进入这个桶： $\mathbb{E}[X] = n \cdot \frac{1}{n} = 1$ 。
- 但我们关心的不是平均，而是：

n 个桶里，最倒霉的那个桶会有多满？

负载均衡

- 一个桶装 k 个球有多罕见？
- 固定看一个桶。它恰好收到 k 个球的概率是 $\Pr[X = k] = \binom{n}{k} \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{n-k}$.
- 当 n 很大、 k 不太大时， $\binom{n}{k} \approx \frac{n^k}{k!}$, $\left(1 - \frac{1}{n}\right)^n \approx e^{-1}$
- 所以： $\Pr[X = k] \approx \frac{1}{e k!}$.
- 重点不是 e ，而是：

$\Pr[X = k]$ 大约按 $1/k!$ 快速下降
- 也就是说，单个桶出现很高负载非常罕见

负载均衡

- 期望有 k 个球左右的桶数约为 $n\Pr[X = k] \approx \frac{n}{e k!}$.
- 最大负载出现的临界位置，就是这个量大约等于 1: $\frac{n}{k!} \approx 1$ (忽略常数 e), 即 $k! \approx n$
- 利用 Stirling 公式: $k! \approx \left(\frac{k}{e}\right)^k$, 取对数: $\log(k!) \approx k \log k - k$.
- 于是大致要求 $k \log k \approx \log n$
- 解是 $k \approx \frac{\log n}{\log \log n}$

Power of Two Choices

- **Power of Two Choices:** 每个球随机选两个桶，放入负载较轻的那个。最大负载降为 $O(\log \log n)$ 。

Power of Two Choices:

for each ball:

i = 随机选桶

j = 随机选桶 ($j \neq i$)

将球放入 $\text{load}[i] < \text{load}[j] ? i : j$

One Choice 里，一个桶变得很满，只需要不断“倒霉地被选中”。

Two Choices 不一样：如果一个桶已经很满，想继续把球放进去，另一个随机选到的桶也必须同样很满。

所以高负载桶的比例**每升一级**都会近似平方： $p_k = \Pr[\text{桶负载} \geq k]$

$$p_{k+1} \approx p_k^2$$

平方、再平方、再平方，很快就掉到 $1/n$ ，因此最大负载只增长到 $O(\log \log n)$

随机化思想的应用

- 差分隐私
- 随机奇异值分解

为什么需要 “差分隐私”

- 目的： 对一个查询，隐藏精确答案，但保留统计规律
- 为什么精确答案也会泄露隐私？
- 例子：
 - 数据库里有 1000 人
 - 查询：患某疾病人数 = 237
 - Alice 离开数据库后再查询：236
- 马上能推出：Alice 患有该疾病。

差分隐私的核心思想之一：引入随机性

- 问题：“你是否做过某件敏感行为？”
- 如果要求如实回答，隐私风险很高。
 - 于是设计 **随机化的响应**：
 - 以 $1/2$ 概率如实回答；
 - 以 $1/2$ 概率随机回答 Yes / No。
- 随机性的作用：

让单次输出具有“不确定性”

随机化到底保护了什么？

- 假设真实答案是 Yes: $P(\text{输出 Yes} \mid \text{真实 Yes}) = \frac{1}{2} + \frac{1}{2} * \frac{1}{2} = \frac{3}{4}$
- 如果真实答案是 No: $P(\text{输出 Yes} \mid \text{真实 No}) = \frac{1}{2} * \frac{1}{2} = \frac{1}{4}$
- 于是攻击者不能确定: 这个人的真实信息到底是什么
- 但同时, **大量用户一起回答时, 仍然可以统计出总体比例**

从随机化到差分隐私

$$P(Y = \text{Yes} \mid X = \text{Yes}) = \frac{3}{4}$$

$$P(Y = \text{Yes} \mid X = \text{No}) = \frac{1}{4}$$

所以同样看到 Yes:

$$\frac{P(Y = \text{Yes} \mid X = \text{Yes})}{P(Y = \text{Yes} \mid X = \text{No})} = 3.$$



对于任意两个可能输入 x, x' ，以及任意输出 y :

$$\Pr[R(x) = y] \leq e^\epsilon \Pr[R(x') = y].$$

得到: $e^\epsilon = 3$,

$$\boxed{\epsilon = \ln 3}$$

从随机化到差分隐私：推广到数据集

- 假设两个数据集 D, D' ，只差一个人的数据，一个随机算法 M 应该满足：

$$\Pr[M(D) \in S] \approx \Pr[M(D') \in S]$$

- S ：任意输出事件
- ϵ ：隐私强度
- 也就是说：一个用户加入或离开数据集后，算法的输出分布不应该发生太大变化。
- 写成： $\Pr[M(D) \in S] \leq e^\epsilon \Pr[M(D') \in S]$.
- ϵ 控制两个输出分布最多能差多少
 - ϵ 小：两个分布很接近，隐私强；
 - ϵ 大：差异更明显，隐私弱。

Toy Example

- “你上课有没有刷过短视频？”，“你有没有逃过算法设计与分析课程？”
- 设全班真实做过事情的同学比例是 p 。一个人最终回答 Yes 的概率（用随机化回答）： $q = p \times \frac{3}{4} + (1 - p) \times \frac{1}{4}$ 。
- 整理： $q = \frac{1}{4} + \frac{1}{2}p$ ，所以： $p = 2q - \frac{1}{2}$
- 例如 100 个学生里，有 **55** 人回答 **Yes**： $q = 0.55$
- 那么估计真实比例： $p = 2 \times 0.55 - 0.5 = 0.60$

奇异值分解 (SVD)

- SVD 到底是什么？
- 任意矩阵 $A \in \mathbb{R}^{m \times n}$ ，都可以写成： $A = U\Sigma V^T$
 - V ：输入空间中的重要方向；
 - Σ ：每个方向的重要程度；
 - U ：这些方向经过 A 后对应的输出方向。

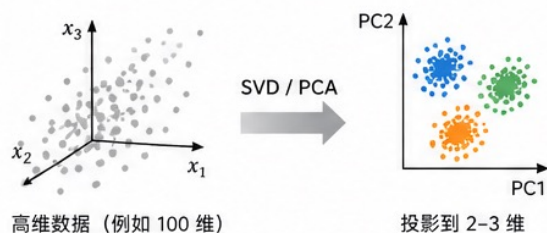
为什么需要奇异值分解

- 真正重要的是奇异值： $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$.
- 如果： $\sigma_1, \sigma_2, \sigma_3$ 很大，而后面的奇异值很小，那么说明：这个高维矩阵真正重要的信息，其实集中在少数几个方向。
- 于是： $A = \sum_{i=1}^r \sigma_i u_i v_i^T$ 可以近似成： $A_k = \sum_{i=1}^k \sigma_i u_i v_i^T$ 其中 $k \ll r$ (r 是 矩阵的秩)
- 也就是**用一个低秩矩阵近似原来的大矩阵**

SVD的应用

1 降维与 PCA（数据分析）

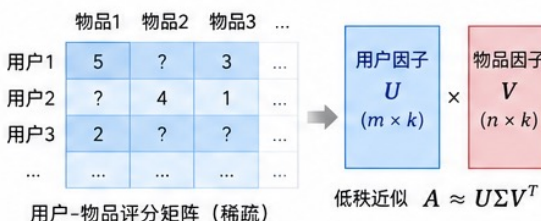
用少数几个主方向表示高维数据，去除冗余，可视化和分类更容易。



价值：更好的可视化、去噪、特征提取，常用于分类、聚类等下游任务。

2 推荐系统（矩阵分解）

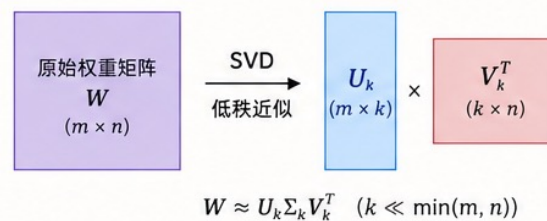
用户-物品评分矩阵通常稀疏且近似低秩，SVD 用少量潜在因子表示用户和物品。



价值：预测缺失的评分，进行个性化推荐，广泛用于电商、视频、新闻等领域。

3 模型压缩与加速（深度学习）

神经网络中的大权重矩阵往往存在低秩结构，用低秩近似可以减少参数量和计算量。



价值：减少模型参数、降低内存和计算成本，用于模型压缩、推理加速（如大模型、LoRA 等）。

4 去噪与图像处理

图像/信号中的主要结构集中在前几个奇异值，去掉小的奇异值可以去除噪声。



价值：图像去噪、压缩、超分辨、视频处理、医学影像、信号恢复等。

5 自然语言处理（文本语义）

用低秩分解在词-文档矩阵中提取潜在语义，用于主题发现和语义表示。



价值：主题建模（如 LSA）、文本相似度、信息检索、文本聚类与可视化。

6 其他应用

- 图机器学习**
图的邻接矩阵低秩近似、谱聚类、节点表示学习。
- 科学计算与仿真**
降维、模型降阶、加速大规模数值计算。
- 异常检测**
用重构误差检测异常数据（低秩 + 稀疏分解）。
- 多模态与表示学习**
在图像、语音、文本等多模态数据中提取共享结构。
- 信息检索**
用低秩向量表示加速相似度搜索（如矩阵分解 + ANN）。

SVD 的核心成分

- **U 矩阵**: 大小为 $m \times m$, 其列向量是 AA^T 的特征向量, 称为**左奇异向量**。
- **Σ 矩阵**: 大小为 $m \times n$, 对角线上的元素 σ_i 称为**奇异值**, 其余元素为 0。
- **V 矩阵**: 大小为 $n \times n$, 其列向量是 $A^T A$ 的特征向量, 称为**右奇异向量**

经典 dense SVD (以 $A \in \mathbb{R}^{m \times n}$, $m \geq n$ 为例) 可以简洁概括为:

1. 双对角化

1. 用 Householder 变换把 A 化为双对角矩阵: $A = Q_L B Q_R^T$

2. 其中 B 是双对角矩阵。

1. 复杂度约: $O(mn^2)$

3. 计算双对角矩阵 B 的 **SVD**, $B = \tilde{U} \Sigma \tilde{V}^T$

1. 用 QR Iteration, 复杂度约: $O(n^3)$

4. 恢复原矩阵的左右奇异向量 $U = Q_L \tilde{U}, V = Q_R \tilde{V}$

5. 得到最终 **SVD** $A = U \Sigma V^T$

双对角化: **Householder**变换算法

- Householder 变换算法可以精简成:

- 给定向量 $x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$

- 设目标方向 $\alpha = \pm \|x\|, e_1 = [1, 0, \dots, 0]^T$

- 构造 Householder 向量 $v = x - \alpha e_1$

- 构造反射矩阵 (正交矩阵) $H = I - 2 \frac{vv^T}{v^T v}$

- 作用于 x : $Hx = \alpha e_1$

因为 Householder 矩阵是对称的:

$$G^T = G,$$

如果

$$Gx = \alpha e_1,$$

转置一下就是

$$x^T G = \alpha e_1^T.$$

双对角化例子

$$A = \begin{bmatrix} 6 & 17 \\ 8 & 6 \\ 0 & 0 \end{bmatrix}$$

双对角化 A

$$\begin{bmatrix} 6 \\ 8 \\ 0 \end{bmatrix}, \left\| \begin{bmatrix} 6 \\ 8 \\ 0 \end{bmatrix} \right\| = 10.$$

构造 Householder 矩阵:

$$Q_L = \begin{bmatrix} \frac{3}{5} & \frac{4}{5} & 0 \\ \frac{4}{5} & -\frac{3}{5} & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

左乘: $B = Q_L^T A$.

这里 $Q_L^T = Q_L$, 于是

$$B = \begin{bmatrix} \frac{3}{5} & \frac{4}{5} & 0 \\ \frac{4}{5} & -\frac{3}{5} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 6 & 17 \\ 8 & 6 \\ 0 & 0 \end{bmatrix}$$

得到

$$B = \begin{bmatrix} 10 & 15 \\ 0 & 10 \\ 0 & 0 \end{bmatrix}$$

已经是双对角矩阵。

这个例子里不需要右边 Householder, 所以

$$Q_R = I$$

双对角化的时间复杂度

- 左 Householder: 消第 i 列下面 $\begin{bmatrix} * \\ * \\ \vdots \\ * \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \\ \vdots \\ 0 \end{bmatrix}$
- 右 Householder: 消第 i 行右边 $[* \ * \ * \cdots] \rightarrow [* \ * \ 0 \cdots]$
- 第 i 轮操作的还是一个大约 $(m-i) \times (n-i)$ 的剩余子矩阵, 因此成本大约是 $O((m-i)(n-i))$.
- 一共做约 n 轮, 所以 $\sum_{i=1}^n O((m-i)(n-i))$
- 展开后主导项就是 $\boxed{O(mn^2)}$.

求矩阵的特征值和特征向量

- 给定矩阵 A ，计算特征值和特征向量分两步：
- 求特征值
 - $Av = \lambda v$ 得到 $(A - \lambda I)v = 0$ ，要有非零解 v ，必须满足 $\det(A - \lambda I) = 0$
 - 解这个方程得到特征值 λ
- 求特征向量
 - 把每个特征值 λ ，代回 $(A - \lambda I)v = 0$
 - 求非零解 v ，就是对应的特征向量。

Givens Rotation 和 implicit QR

- 要处理两个数 a, b , 目标是 $\begin{bmatrix} a \\ b \end{bmatrix} \rightarrow \begin{bmatrix} r \\ 0 \end{bmatrix}$ 其中 $r = \sqrt{a^2 + b^2}$ 取 $c = \frac{a}{r}, \quad s = \frac{b}{r}$ 构造 $G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$
- 于是

$$G \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix}$$

- 设 $B = \begin{bmatrix} d_1 & e_1 & & \\ & d_2 & e_2 & \\ & & \ddots & e_{n-1} \\ & & & d_n \end{bmatrix}$, implicit QR 通常用 $B^T B$ 右下角 2×2 块的特征值作为 **shift**, 但这个小小

块可以直接由 (d_i, e_i) 得到: $\begin{bmatrix} d_{n-1}^2 + e_{n-2}^2 & d_{n-1}e_{n-1} \\ d_{n-1}e_{n-1} & d_n^2 + e_{n-1}^2 \end{bmatrix}$ 然后求这个 2×2 矩阵的两个特征值, 选择更

接近右下角元素的那个作为 shift, 这就是常见的 Wilkinson shift。

双对角矩阵求SVD

- 设 $B = \begin{bmatrix} d_1 & e_1 & & \\ & d_2 & e_2 & \\ & & \ddots & e_{n-1} \\ & & & d_n \end{bmatrix}$
- 目标是求 $B = U\Sigma V^T$.

- 从双对角矩阵 B 出发；
- 选择 *shift*: μ
- 根据 $B^T B - \mu I$ 隐式确定第一个右 Givens rotation
- 交替进行右、左 Givens rotation, 进行 bulge chasing
- 最终 B 收敛成对角矩阵: $B \rightarrow \Sigma = \begin{bmatrix} \sigma_1 & & \\ & \sigma_2 & \\ & & \ddots \end{bmatrix}$
- 累积过程中所有左右旋转, 得到 U, V 。
- 最终得到: $B = U\Sigma V^T$

SVD求解

给定 $B = \begin{bmatrix} 10 & 15 \\ 0 & 10 \\ 0 & 0 \end{bmatrix}$ 只看有效的 2×2 块: $\begin{bmatrix} 10 & 15 \\ 0 & 10 \end{bmatrix}$

1. 计算特征值: 25, 400.

2. $\begin{bmatrix} d_1^2 & d_1 e_1 \\ d_1 e_1 & d_2^2 + e_1^2 \end{bmatrix} = \begin{bmatrix} 100 & 150 \\ 150 & 325 \end{bmatrix}$, 取其靠近右下角的特征值作为 shift: $\mu = 400$

3. 右 Givens rotation

由 $f = 10^2 - \mu = -300, g = 10 \times 15 = 150$ 构造右旋转

$$G_R = \frac{1}{\sqrt{5}} \begin{bmatrix} -2 & -1 \\ 1 & -2 \end{bmatrix}.$$

右乘:

$$BG_R = \frac{1}{\sqrt{5}} \begin{bmatrix} -5 & -40 \\ 10 & -20 \\ 0 & 0 \end{bmatrix}$$

这时产生了左下角的 bulge。

4. 左 Givens rotation 消 bulge

对第一列 $\begin{bmatrix} -5 \\ 10 \end{bmatrix}$ 构造左旋转:

$$G_L^T = \frac{1}{\sqrt{5}} \begin{bmatrix} -1 & 2 \\ -2 & -1 \end{bmatrix} \text{ 于是 } G_L^T B G_R = \begin{bmatrix} 5 & 0 \\ 0 & 20 \\ 0 & 0 \end{bmatrix}, \text{ 已经对角化。}$$

5. 得到奇异值

重新按从大到小排列: $\Sigma = \begin{bmatrix} 20 & 0 \\ 0 & 5 \\ 0 & 0 \end{bmatrix}$ 所以 $\sigma_1 = 20, \sigma_2 = 5$

6. 得到 U, V QR 过程中:

- 所有左 Givens 累积起来 $\rightarrow U$
- 所有右 Givens 累积起来 $\rightarrow V$

因此 $B = U \Sigma V^T$

SVD求解复杂度分析

消除一个元素是一条链：

$$\begin{array}{ccccccc} & & \begin{bmatrix} * & * & 0 & 0 \\ 0 & * & * & 0 \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} & \rightarrow & \begin{bmatrix} * & * & 0 & 0 \\ * & * & * & 0 \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} & \rightarrow & \begin{bmatrix} * & * & * & 0 \\ 0 & * & * & 0 \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} \\ \rightarrow & \begin{bmatrix} * & * & 0 & 0 \\ 0 & * & * & 0 \\ 0 & * & * & * \\ 0 & 0 & 0 & * \end{bmatrix} & \rightarrow & \begin{bmatrix} * & * & 0 & 0 \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} & \rightarrow & \begin{bmatrix} * & * & 0 & 0 \\ 0 & * & * & 0 \\ 0 & 0 & * & * \\ 0 & 0 & * & * \end{bmatrix} & \rightarrow & \begin{bmatrix} * & * & 0 & 0 \\ 0 & * & * & 0 \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} \end{array}$$

其中：

*= 本轮希望逐步压小的副对角元素

*= bulge

一次 **sweep** 要沿整个双对角线走一遍，所以成本是 $O(n)$ 。

当最下面的一个奇异值收敛后，就可以做 deflation，把问题规模从 n 降到 $n-1$ ；然后继续处理剩下的部分。

于是总成本近似是： $O(n) + O(n-1) + O(n-2) + \dots + O(1) = O(\sum_{k=1}^n k) = O(n^2)$

SVD求解复杂度

- 整个 implicit QR 过程中，总共大约有 $O(n^2)$ 次 Givens rotation。
- 每得到一个左 Givens G_L ，需要更新 $U \leftarrow UG_L$ 由于 G_L 只影响 U 的两列，但每列长度为 n ，所以一次更新成本： $O(n)$
- 同理， $V \leftarrow VG_R$ 一次也是 $O(n)$
- 因此总成本： $O(n^2)$ 次旋转 $\times O(n)$ 每次更新 = $O(n^3)$

随机化SVD

- 不在原始 $m \times n$ 大矩阵上做完整SVD，而是先压缩到 $m \times k$ 的低维子空间，再做SVD
 - 随机生成一个小矩阵 $\Omega \in \mathbb{R}^{n \times k}$ ，随机投影 $Y = A\Omega$ ，维度变成 $Y \in \mathbb{R}^{m \times k}$ ，成本： $O(mnk)$
 - 对 Y 做QR分解，得到正交基 Q ，计算 $Y = QR$ ， $Q \in \mathbb{R}^{m \times k}$ ，成本： $O(mk^2)$
 - 把原始大矩阵投影到这个低维空间 $B = Q^T A$ ，此时 $B \in \mathbb{R}^{k \times n}$ 成本： $O(mnk)$
 - 求左右奇异向量成本： $O(k^2)$ 次旋转 $\times O(n)$ 每次更新 = $O(k^3)$
- 所以总复杂度大约： $O(mnk + (m + n)k^2)$ ，当 $k \ll n$ 时，主导项就是 $O(mnk)$

正确性分析

- 先从随机投影开始: $Y = A\Omega = U\Sigma V^T\Omega$
- 其中 Ω 是随机矩阵。把奇异方向按“前 k 个主要方向”和“其余方向”拆开: $U = [U_k \ U_\perp]$
- 于是 $Y = U_k\Sigma_k\Omega_1 + U_\perp\Sigma_\perp\Omega_2$ 由于前 k 个奇异值通常更大, $\Sigma_k \gg \Sigma_\perp$
- 因此 Y 中主要由前 k 个左奇异方向贡献: $Y \approx U_k\Sigma_k\Omega_1$
- 也就是说, Y 的列空间会接近 A 的主要左奇异子空间: $\text{span}(Y) \approx \text{span}(U_k)$
- 对 Y 做 QR 分解: $Y = QR$, QR 不改变列空间, 因此 $\text{span}(Q) = \text{span}(Y) \approx \text{span}(U_k)$, 所以 Q 可以作为 A 主要列空间的一组近似正交基。
 - 随机向量经过 A 之后, 会被 A 在不同奇异方向上按奇异值大小进行不同程度的放大。大的奇异方向自然会在结果里占主导。

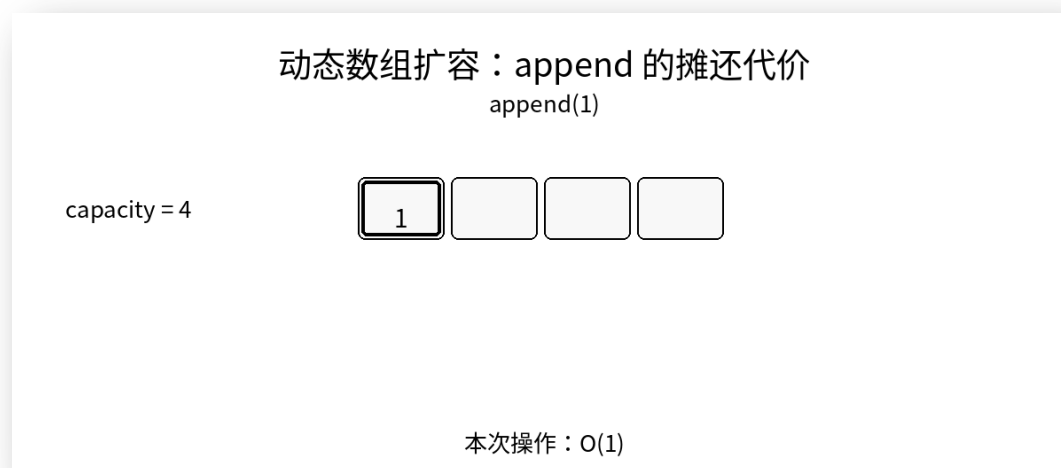
正确性分析

- 接下来把 A 投影到这个低维空间: $B = Q^T A$, 由于 Q 已经覆盖了 A 的主要信息, $A \approx QQ^T A = QB$, 然后对小矩阵 B 做精确 SVD: $B = \tilde{U}\Sigma V^T$
- 代回: $A \approx QB = Q\tilde{U}\Sigma V^T$
- 令 $U = Q\tilde{U}$, 最终得到: $A \approx U\Sigma V^T$
- 所以整条逻辑可以概括为:

随机采样 $\rightarrow$ 找到主要子空间 $\rightarrow$ 投影降维 $\rightarrow$ 在小矩阵上做 SVD $\rightarrow$ 映射回原空间

从“单次最坏情况”到“一系列操作”

- 到目前为止，我们通常分析：
 - 一个算法输入规模为 n 时，需要多少时间？
 - 一次操作的最坏时间是多少？
- 但很多数据结构的操作成本并不均匀，有些操作偶尔非常昂贵，但大部分操作都很便宜



- 类似“充值返利/买十送一”：单次贵，但摊到多次使用就便宜。

均摊分析/摊还分析：关注一系列操作的总成本

- 假设连续执行 m 次操作，传统思路关注：
 - 每一次操作最坏需要多少时间？
- 均摊分析关注：
 - 这 m 次操作一共需要多少时间？
- 如果总时间是 $O(m)$ ，那么我们可以说：每次操作的摊还时间复杂度为 $O(1)$

概念	在问什么
最坏情况分析	某一次操作最坏有多贵？
平均情况分析	对随机输入，平均有多贵？
均摊分析	对任意一串操作，平均到每次有多贵？

均摊分析的方法

聚合法 (Aggregate)

- 先计算连续 m 次操作的总成本 $T(m)$
- 再平均到每次操作: $T(m)/m$
- 思想: “不看某一次有多贵, 看一整串操作总共多贵。”

均摊分析的方法

记账法 (Accounting)

- 给每次操作规定一个摊还费用 $\hat{c}_i$
- 便宜操作可以“多收一点”，把余额存起来
- 将来遇到昂贵操作时，用之前积累的余额支付
- 要保证任何时刻余额不为负
- 思想：“现在多付一点，为以后买单。”

均摊分析的方法

势能法 (Potential)

- 定义数据结构当前状态的势能 Φ ，表示系统中“储存的能量”
- 第 i 次操作的摊还成本：

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$$

- c_i ：这次操作真实发生的成本
- $\Phi_i - \Phi_{i-1}$ ：这次操作导致系统“储能”的变化
- 通常要求初始势能为 0，并保证势能非负
- 思想：“不仅看这次花了多少，还看系统积攒/释放了多少能量。”

聚合法

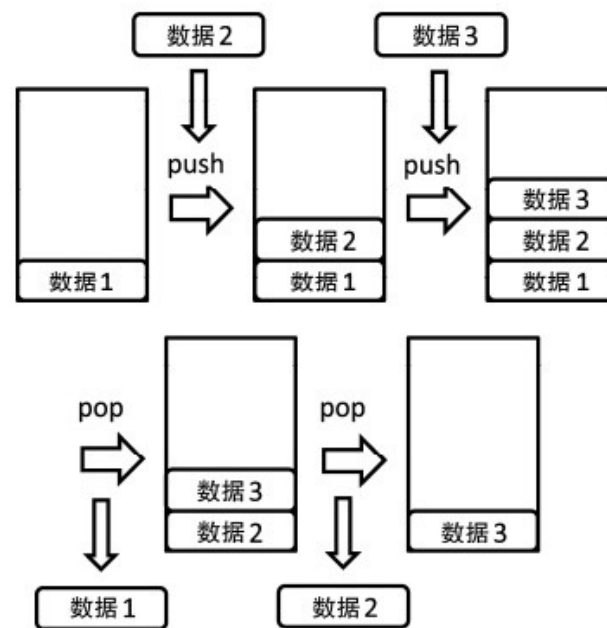
- 聚合法的目的：用于分析一系列操作的摊还代价
- 考虑连续执行 n 个操作： $op_1, op_2, \dots, op_n$ ，设第 i 个操作的实际代价为 t_i ，计算整个操作序列的总代价：

$$T(n) = t_1 + t_2 + \dots + t_n$$

- 求出总代价的上界 $T(n)$
- 将总成本平均到 n 个操作： $\frac{T(n)}{n}$ ，得到每个操作的摊还代价上界
- 特点
 - 所有操作被赋予相同的摊还代价
 - 不需要区分不同操作如何“收费”
 - 关键在于：如何计算整个操作序列的总代价
 - 通常是三种摊还分析方法中最直观的一种

普通栈操作及其时间代价

- $\text{Push}(S, x)$: 将对象 x 压入栈 S
- $\text{Pop}(S)$: 弹出并返回 S 的顶端元素
- 两个操作的时间复杂度都是 $O(1)$
- 把每个操作的实际代价视为1
- n 个Push和Pop操作系列的总代价是 n
- n 个操作的实际时间复杂度为 $O(n)$



队列的操作

Multipop操作及其时间代价

- 操作Multipop(S, k):
 - 去掉S的k个顶对象, 当 $|S| < k$ 时弹出整个栈
- 实现算法

```
def multipop(S, k):  
    while S and k > 0:  
        S.pop()  
        k -= 1
```

- Multipop(S, k)的实际代价 (设Pop的代价为1)
 - Multipop的代价为 $\min(|S|, k)$

栈操作序列的均摊分析

- n 个栈操作序列由Push、Pop和Multipop组成
- 粗略分析（有问题）
 - 最坏情况下，每个操作都是Multipop
 - 每个Multipop的代价最坏是 $O(n)$
 - 操作序列的最坏代价为 $T(n) = O(n^2)$
 - 均摊代价为 $T(n)/n = O(n)$
- 精细分析
 - 一个对象在每次被压入栈后至多被弹出一次
 - 在非空栈上调用Pop的次数(包括在Multipop内的调用)至多为Push执行的次数, 即至多为 n
 - 最坏情况下操作序列的代价为 $T(n) \leq 2n = O(n)$
 - 均摊代价 = $T(n)/n = O(1)$

二进制计数器

- 由0开始计数的 k 位二进制计数器
- 输入： k 位二进制变量 x ，初始值为0
- 输出： $x + 1$ （二进制）
- 数据结构：
 - $A[0..k - 1]$ 作为计数器, 存储 x
 - x 的最低位在 $A[0]$ 中, 最高位在 $A[k - 1]$ 中
 - $x = \sum_{i=0}^{k-1} A[i] * 2^i$

二进制
数字

1 1 0 0 1 0 1

实际数字 2^6 2^5 2^4 2^3 2^2 2^1 2^0

计算方法：将二进制数中值为1所代表的实际数字加起来即可

计数器加1算法

- 输入: $A[0..k-1]$, 存储二进制数 x
- 输出: $A[0..k-1]$, 存储二进制数 $x + 1 \bmod 2^k$

INCREMENT(A)

$i \leftarrow 0$

while $i < k$ **and** $A[i] = 1$

$A[i] \leftarrow 0$

$i \leftarrow i + 1$

if $i < k$

$A[i] \leftarrow 1$

INCREMENT操作分析

Counter Value (N)	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	每次操作代价 / Cost
0	0	0	0	0	0	0	0	0	—
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	2
3	0	0	0	0	0	0	1	1	1
4	0	0	0	0	0	1	0	0	3
5	0	0	0	0	0	1	0	1	1
6	0	0	0	0	0	1	1	0	2
7	0	0	0	0	0	1	1	1	1
8	0	0	0	0	1	0	0	0	4
9	0	0	0	0	1	0	0	1	1
10	0	0	0	0	1	0	1	0	2
11	0	0	0	0	1	0	1	1	1
12	0	0	0	0	1	1	0	0	3
13	0	0	0	0	1	1	0	1	1
14	0	0	0	0	1	1	1	0	2
15	0	0	0	0	1	1	1	1	1
16	0	0	0	1	0	0	0	0	5

INCREMENT操作分析

Counter Value N	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total Cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

均摊分析

- 精细分析

- $A[0]$ 每1次操作发生一次改变, 总次数为 n
- $A[1]$ 每2次操作发生一次改变, 总次数为 $n/2$
- $A[2]$ 每4次操作发生一次改变, 总次数为 $n/2^2$
- $A[3]$ 每8次操作发生一次改变, 总次数为 $n/2^3$

- 对于 $i = 0, 1, \dots, \lg n$, $A[i]$ 改变次数为 $n/2^i$

- 对于 $i > \lg n$, $A[i]$ 不发生改变 (从 0 开始执行 n 次 INCREMENT 后, 计数器最大值为 n , 因此只可能涉及 $A[0]$ 到 $A[\lfloor \lg n \rfloor]$ 。对于 $i > \lg n$, $A[i]$ 始终为 0, 不会发生翻转)
- $T(n) = \sum_{i=0}^{\lg n} n/2^i < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n = O(n)$

- 每个Increment操作的均摊代价为 $O(1)$

从聚合法到记账法

- 聚合法从整体出发：先计算一系列操作的总成本，再平均到每个操作
- 但现实中往往包含多种操作：有些操作很便宜，有些操作偶尔很贵
- 因此引入记账法（**Accounting Method**）：给不同操作规定不同的“收费”
- 便宜操作多收一点，存为信用
- 昂贵操作发生时，用之前积累的信用支付
- 于是：

$$\text{摊还代价} = \text{实际代价} + \text{信用余额的增加量}$$

- 如果余额减少，那么增加量就是负数。

记账法的思想

- 一个操作序列中可能包含不同类型的操作
- 不同类型操作的**实际代价**可能不同
- 记账法为不同操作分配不同的**摊还代价**
 - 摊还代价可以高于实际代价，也可以低于实际代价
 - 若某次操作的摊还代价 $>$ 实际代价：多收的部分作为**信用**保存，用于支付未来的昂贵操作
 - 若某次操作的摊还代价 $<$ 实际代价：不足的部分由之前积累的**信用**支付
- 要保证任何时候**累计信用**都不能为负

核心思想：便宜时“多收钱”，昂贵时“用存款”。

记账法的基本原理

- 设 c_i 为第 i 次操作的**实际代价**
- 设 α_i 为第 i 次操作的**摊还代价**
- 摊还代价高于实际代价的部分，作为**信用**保存
- 之后昂贵操作可以使用之前积累的信用
- 任何时刻，**累计信用**必须非负

$$\underbrace{\sum_{i=1}^n \alpha_i}_{\text{累计收取}} - \underbrace{\sum_{i=1}^n c_i}_{\text{实际支出}} \geq 0$$

- 因此：**累计摊还代价**始终是**累计实际代价**的上界。

栈操作序列的分析

- 各栈操作的实际代价
 - $\text{Cost}(\text{PUSH})=1$
 - $\text{Cost}(\text{POP})=1$
 - $\text{Cost}(\text{MULTIPOP})=\min\{k, s\}$
- 各栈操作的摊还代价
 - $\text{Cost}(\text{PUSH})=2$
 - 一个1用来支付PUSH的开销,
 - 另一个1存储在压入栈的元素上, 预支POP的开销
 - $\text{Cost}(\text{POP})=0$
 - $\text{Cost}(\text{MULTIPOP})=0$

一个元素一生只会 PUSH 一次、POP 一次。PUSH 时把一生的费用都交了

栈操作序列的分析

- 摊还代价满足： $\sum_{i=1}^n \alpha_i - \sum_{i=1}^n c_i \geq 0$ 对于任意 n 个操作序列都成立
- 因为在 n 个操作序列中, POP个数(包括MULTIPOP中的 POP)不大于PUSH个数
- n 个栈操作的总摊还代价为 $O(n)$, 因此, 每个栈操作的摊还代价为 $O(1)$

二进制计数器操作序列分析

- Increment操作的均摊代价
- 每次一位被置1时，付2元
 - 1元用于置1的开销
 - 1元存储在该位上，用于支付其被置0时的开销
 - 置0操作无需再付款
- $\text{Cost}(\text{Increment})=2$
- 均摊代价满足
 - $\sum_{i=1}^n \alpha_i - \sum_{i=1}^n c_i \geq 0$ 对于任意n个操作序列都成立，从前面的分析可知 $T(n) < 2n$
- n个Increment操作序列的总均摊代价 $= O(n)$
- 每个Increment操作的均摊代价为 $O(1)$

从记账法到势能法

- 在记账法中：
 - PUSH 多收 1 个单位
 - 这个信用“存”在新压入的元素上
 - 将来 POP 时，用这个元素上的信用支付
- 如果我们只关心“现在一共存了多少钱”，还需要追踪每一块钱属于哪个元素吗？

把整个数据结构(D)当前积累的信用，用一个函数表示 $\phi(D)$ ，称为势能函数（Potential Function）

从记账法到势能法

- 在记账法中，当某次操作的摊还代价大于实际代价时，多收取的部分作为信用保存，并可与某个数据对象相关联
- 势能法不再分别记录这些信用，而是用一个势能函数 $\Phi(D)$ 描述整个数据结构当前积累的“信用”
- 当某次操作的摊还代价 大于 实际代价时，说明有一部分成本被“存起来”，因此势能增加
- 当某次操作的摊还代价 小于 实际代价时，说明使用了之前积累的信用，因此势能减少

数据结构势能的定义

- 考虑从初始状态 D_0 开始执行 n 个操作
- 对于第 i 个操作：
 - 实际代价为 c_i
 - 操作将数据结构从 D_{i-1} 变为 D_i
 - $\phi(D_i)$ 表示状态 D_i 的势能
- 定义第 i 个操作的摊还代价： $\alpha_i = c_i + \phi(D_i) - \phi(D_{i-1})$
- 通常选择势能函数满足： $\phi(D_0) = 0, \phi(D_i) \geq 0$
- 于是： $\sum_{i=1}^n \alpha_i = \sum_{i=1}^n c_i + \phi(D_n) - \phi(D_0)$
- 因此： $\sum_{i=1}^n \alpha_i \geq \sum_{i=1}^n c_i$
- 也就是：总摊还代价是总实际代价的上界。

势能的解读

- 想象做算法分析时开了一个“系统账户”。
- 每次操作既有实际代价，也可以多收一点存入账户，或从账户中取出之前存的钱（记账法）。
- **账户余额用势能 $\Phi(D)$ 表示**，并要求始终满足

$$\Phi(D_i) \geq 0, \Phi(D_0) = 0.$$

- 因此，对任意一段操作序列，均摊总代价不会低于实际总代价。

直觉说法	数学式子
每次操作：实际花的钱 + 存入/取出余额	$(\hat{c}_i = c_i + [\phi(D_i) - \phi(D_{i-1})])$
所有操作的总账户情况	$\left(\sum_i \hat{c}_i = \sum_i c_i + [\phi(D_n) - \phi(D_0)]\right)$
保证账户余额不为负	$(\phi(D_n) \geq \phi(D_0))$
初始余额为零	$(\phi(D_0) = 0)$
均摊总代价 $\geq$ 实际总代价	$\left(\sum c_i \leq \sum \hat{c}_i\right)$

栈操作序列的分析

- 栈的势能定义
- $\phi(D_i)$ 为栈 D_i 中对象的个数
 - $\phi(D_0) = 0$, D_0 是空栈
 - $\phi(D_i) \geq 0 = \phi(D_0)$, 因为栈中对象个数不会小于0
 - n 个操作的总均摊代价是实际 代价的上界
- 栈操作的摊还代价（设栈 D_{i-1} 中具有 s 个对象）
 - PUSH: $\alpha_i = c_i + \phi(D_i) - \phi(D_{i-1}) = 1 + (s + 1) - s = 2$
 - POP: $\alpha_i = c_i + \phi(D_i) - \phi(D_{i-1}) = 1 + (s - 1) - s = 0$
 - MULTIPOP: 设 $k' = \min(k, s)$, $\alpha_i = c_i + \phi(D_i) - \phi(D_{i-1}) = k' + (s - k') - s = 0$
 - n 个栈操作序列的平摊代价是 $-- O(n)$

“想势能函数”的一般逻辑是什么？

- 先看哪个操作贵，再问：“这个贵操作消耗掉了什么之前积累起来的東西？”

昂贵操作：MULTIPOP



为什么能贵？



因为栈里积累了很多元素



这些元素从哪来的？



以前的 PUSH



所以用“栈中元素数量”衡量积累程度



$\Phi(D)$ = 栈大小

势能函数不是去描述“现在花了多少钱”，而是描述
“当前数据结构里还积攒着多少未来可能要付的钱”。

二进制计数器操作序列分析

- 先找“昂贵操作为什么会贵”，再找它消耗了什么过去积累的状态。
- 为什么某一次 INCREMENT 能一下翻这么多位？因为它前面已经积累了很多连续的 1。
- 比如：0111
- 自然想到： $\Phi(D) =$ 计数器中 1 的个数
- 计数器的势能定义
 - $\phi(D_i)$ 定义为第 i 个操作后计数器中 1 的个数 b_i
 - $\phi(D_0) = 0$ ， D_0 中 1 的个数为 0
 - $\phi(D_i) \geq 0 = \phi(D_0)$ ，因为计数器中 1 的个数不会小于 0

二进制计数器操作序列分析

- 势能定义: $\phi(D_i)$ = 当前 k 位计数器中 1 的个数, t_i : 第 t_i 次 INCREMENT 前, 计数器最低位开始连续的 1 的个数。
- 实际代价: $c_i = t_i + 1$ (一般情况), $c_i = k$ (溢出时).
- 势能变化

$$\Delta\phi = \begin{cases} 1 - t_i, & \text{非溢出 (产生新 1)} \\ -k, & \text{溢出 (所有 1 清空)} \end{cases}$$

- 均摊代价

$$\alpha_i = c_i + \Delta\phi = \begin{cases} (t_i + 1) + (1 - t_i) = 2, & \text{非溢出} \\ k + (-k) = 0, & \text{溢出} \end{cases}$$

- 结论:

$$0 \leq \alpha_i \leq 2, \sum_{i=1}^n \alpha_i \leq 2n.$$

- 因此总均摊代价仍为 $O(n)$ 。

例如:

1010111 $\rightarrow$ 1011000

111

这里 $t_i = 3$, 实际翻转 $3 + 1 = 4$ 位。

势能变化:

$$\Delta\phi = 1 - t_i$$

因为消失了 t_i 个 1, 又新产生了一个 1。

动态表

- **目标**：使表（数组）尽可能小，但足够大以避免数组溢出
- **问题**：如果我们无法提前知道合适的大小怎么办？
- **解决方案**：动态表
- **思路**：每当表格溢出时，通过malloc或new分配一个新的、更大的表。将所有元素从旧表移动到新表，并释放旧表的存储空间。

动态表的扩张

- 插入一个数组元素时,完成的操作包括
 - 分配一个包含比原表更多的空间的新表
 - 再将原表中的各项复制到新表中去
- 分配一个比原表大一倍的新表
 - 只对表执行插入操作,则表的负载因子总是至少为 $1/2$
 - 浪费掉的空间就始终不会超过表总空间的一半

算法伪码

TABLE-INSERT(T, x)

```
1  if size[T] = 0 then
2    分配大小为 1 的表 table[T]
3    size[T]  $\leftarrow$  1                // ■ 初始化:  $O(1)$ 

4  if num[T] = size[T] then          // ■ 表已满, 触发扩容
5    分配大小为  $2 \times \text{size}[T]$  的新表 new-table
6    将 table[T] 中所有元素复制到 new-table    // ■ 主要开销:  $\theta(\text{num}[T])$ 
7    释放旧表 table[T]
8    table[T]  $\leftarrow$  new-table
9    size[T]  $\leftarrow 2 \times \text{size}[T]$         // ■ 容量翻倍

10 将 x 插入 table[T]                // ■ 普通插入:  $O(1)$ 
11 num[T]  $\leftarrow$  num[T] + 1
```

动态表的实例

动态表：插入与扩容

table[T] size = 1

10

初始容量 = 1

聚合法

- 令 c_i 为第 i 次插入的代价

$$c_i = \begin{cases} i & \text{如果 } i-1 \text{ 是 } 2 \text{ 的倍数} \\ 1 & \text{其他} \end{cases}$$

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	1	1	1	1	1	1	1	1	1
		1	2		4				8	

$$n \text{ 次插入的总开销} = \sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j \leq 3n = O(n)$$

每个动态表操作的均摊代价是 $O(1)$

记账法

- 为第 i 次插入分配一个均摊代价 $\hat{c}_i = 3$ 元
 - 1 元用于支付即时插入的费用
 - 2 元存入信用，用于将来的表扩容
- 当表扩容时，1 元用于移动最近插入的元素，另 1 元用于移动之前插入的旧元素

Example:

\$0 \$0 \$0 \$0 \$2 \$2 \$2 \$2 *overflow*



Example:

 *overflow*



记账法

- 关键不变性：信用余额永远不会低于 0。因此，均摊成本的总和 为实际成本的总和 提供了一个上界。

动态表大小

实际代价

均摊代价

信用余额

i	1	2	3	4	5	6	7	8	9	10
$size_i$	1	2	4	4	8	8	8	8	16	16
c_i	1	2	3	1	5	1	1	1	9	1
$\hat{c}_i$	2*	3	3	3	3	3	3	3	3	3
$bank_i$	1	2	2	4	2	4	6	8	2	4

动态表：势能函数怎么想到？

- 核心问题：扩容很贵，怎样把代价摊到之前的普通插入上？
- 思路：表越接近满，未来扩容的“潜在工作量”越大。
- 因此希望势能满足：
 - 普通插入：表更满 $\rightarrow$ 势能增加，提前“存钱”
 - 发生扩容：容量翻倍 $\rightarrow$ 势能大幅下降，用来支付复制开销
- 定义：

$$\Phi(T) = 2 \text{ num}[T] - \text{size}[T]$$

- 其中， $\text{num}[T]$ 为当前表中元素个数， $\text{size}[T]$ 为表的容量。

摊还分析

设插入前表中有 m 个元素。

① 不扩容

容量不变，只增加一个元素：

$$c_i = 1, \Delta\Phi = 2$$

因此

$$\alpha_i = 1 + 2 = 3$$

也就是：这次实际只花 1，但“多收”2，用来为以后扩容存钱。

② 表满，需要扩容

扩容前：

$$num = size = m$$

扩容并插入后：

$$num = m + 1, size = 2m$$

实际代价：

$$c_i = m + 1$$

势能变化：

$$\Delta\Phi = [2(m + 1) - 2m] - [2m - m] = 2 - m$$

因此：

$$\alpha_i = (m + 1) + (2 - m) = 3$$

所以无论是否扩容：

$$\alpha_i = 3 = O(1)$$

连续 n 次插入：

$$\sum_{i=1}^n \alpha_i \leq 3n = O(n)$$

练习题

- 有两个长度均为 n 的超长文件 A, B 。读取任意指定位置 $A[i]$ 、 $B[i]$ 的代价均为 $O(1)$ 。
- 已知满足以下二者之一：
 - $A = B$
 - A, B 至少有 εn 个位置不同，其中 $0 < \varepsilon < 1$ 。
- 由于 n 很大，不希望逐个位置比较。
- 请设计一个随机化算法判断两个文件是否相同，并回答：
 - 给出算法。
 - 若随机检查 k 个位置，分析算法出错的概率。
 - 若希望错误概率不超过 δ ， k 至少取多大？
 - 该算法属于 Las Vegas 还是 Monte Carlo？
 - 为什么题目必须给出“至少有 εn 个位置不同”这一条件？

参考答案

- (1) 算法

- 随机选取 k 个位置比较。若任一位置不同，则返回“不相同”；否则返回“相同”。

- (2) 错误概率

- 若 $A = B$ ，一定正确。若至少有 εn 个位置不同，则一次未抽到不同位置的概率至多为 $1 - \varepsilon$ 。连续 k 次均未发现差异的概率： $(1 - \varepsilon)^k$

- (3) 要求错误概率不超过 δ

- 令 $(1 - \varepsilon)^k \leq \delta$ 利用 $1 - \varepsilon \leq e^{-\varepsilon}$ ，取 $k \geq \frac{\ln(1/\delta)}{\varepsilon}$ 即可。

- (4) 算法类型

- Monte Carlo。运行时间为 $O(k)$ ，存在小概率误判“相同”。

- (5) 为什么需要 εn 的条件

- 若两个文件只差一个位置，随机抽中该位置的概率仅为 $1/n$ ，可能需要 $\Theta(n)$ 次检查。所以随机采样有效的关键是：差异位置占有一定比例

练习题

- 有 n 个元素，最开始每个元素各自属于一个集合： $\{1\}, \{2\}, \dots, \{n\}$ 。每次选择两个当前存在的集合 A, B 进行合并。合并后， A, B 被替换为集合 $A \cup B$ 。假设集合通过某种数据结构存储，将一个集合中的一个元素移动到另一个集合的代价为 1，其他操作的代价忽略。
- 采用如下 **small-to-large** 策略：
 - 若 $|A| \leq |B|$ ，则将 A 中的所有元素移动到 B 中；若二者大小相等，可任选一个进行移动。
- 经过 $n - 1$ 次合并后，只剩一个包含全部 n 个元素的集合。
- 回答：
 - 单次合并操作的最坏代价是多少？
 - 证明任意一个元素在整个过程中最多被移动 $\lfloor \log_2 n \rfloor$ 次。
 - 证明任意合并序列的总代价为 $O(n \log n)$ 。
 - 构造一个合并序列，使总代价达到 $\Omega(n \log n)$ 。由此给出最坏情况下的紧确界。
 - 如果不采用 small-to-large，而允许任意决定哪个集合移动到另一个集合，证明最坏总代价可以达到 $\Theta(n^2)$ 。

参考答案

- (1) 单次最坏代价

- 若两个集合大小为 $a \leq b$ ，则需移动 a 个元素。最坏时 $a \approx n/2$ ，因此： $\Theta(n)$

- (2) 单个元素最多移动几次

- 若元素 x 被移动时，其所在集合大小为 s ，因为它属于较小集合，合并后新集合大小至少为： $2s$ 因此每移动一次，所在集合大小至少翻倍： $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow n$ 所以每个元素最多移动： $O(\log n)$ 次。

- (3) 总代价

- 共有 n 个元素，每个最多移动 $O(\log n)$ 次，因此总移动次数为： $O(n \log n)$ ，所以每次合并的摊还代价为： $O(\log n)$

- (4) 证明上界是紧的

- 当 $n = 2^k$ 时，每次合并两个相同大小的集合： $1 + 1, 2 + 2, 4 + 4, \dots$ 每一层总移动代价都是 $n/2$ ，共有 $\log n$ 层，因此： $\Theta(n \log n)$

- (5) 不用 **small-to-large**

- 若每次都把大集合搬到小集合中，则总代价可为： $1 + 2 + \dots + (n - 1) = \Theta(n^2)$ 因此： $\Theta(n^2)$

大作业

- 发布在 elearning 上，通过 elearning 提交（有deadline）